

MindSpore Transformers 大模型系列课程

MindSpore Transformers & vLLM推理精度比对

主讲人：Yule

昇思MindSpore研发工程师

MindSpore Transformers SIG Contributor

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers训练性能调优》

目录

- 1. 推理精度比对概念介绍**
- 2. 推理精度比对流程详述**
- 3. 推理精度比对问题定位**

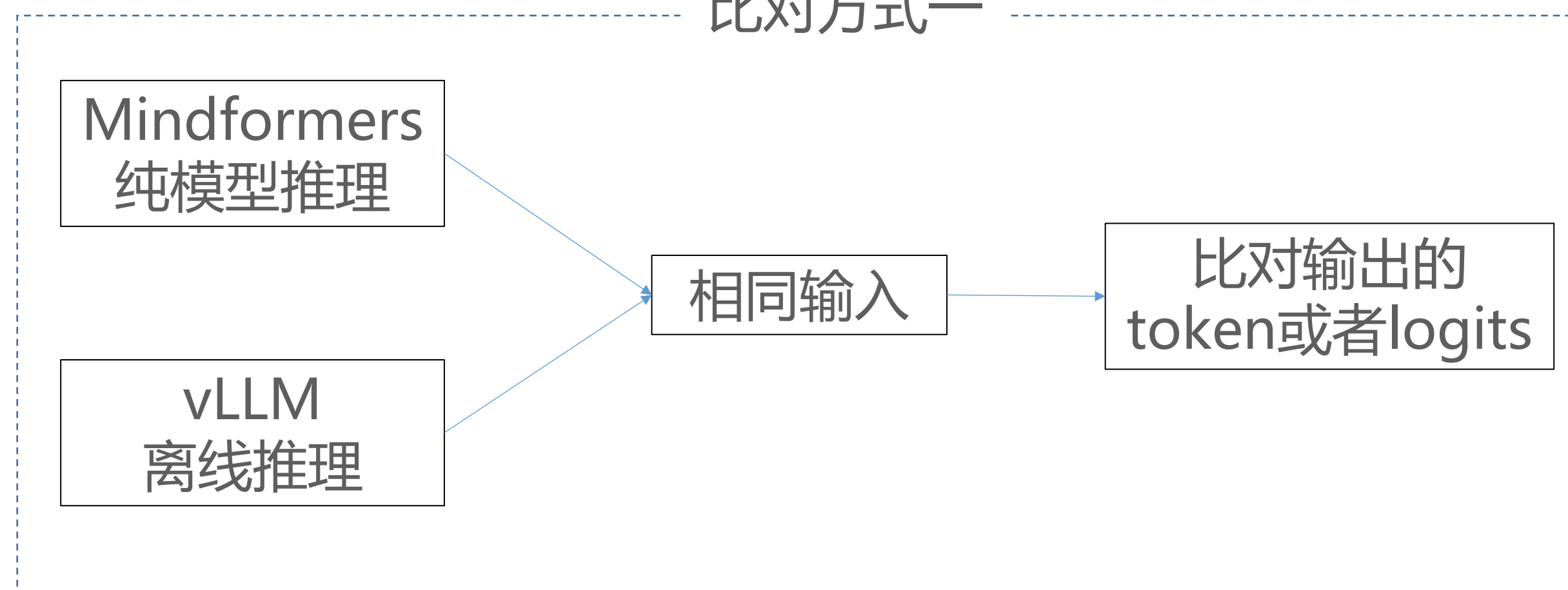
1. 推理精度对比概念介绍

推理精度比对一般在迁移适配新模型或者版本升级时需要，二者对标的标杆也有所不同：

1. 新模型迁移适配时，选择业界主流的vLLM框架作为标杆
2. 版本升级时，可以将验证过精度的旧版本作为标杆

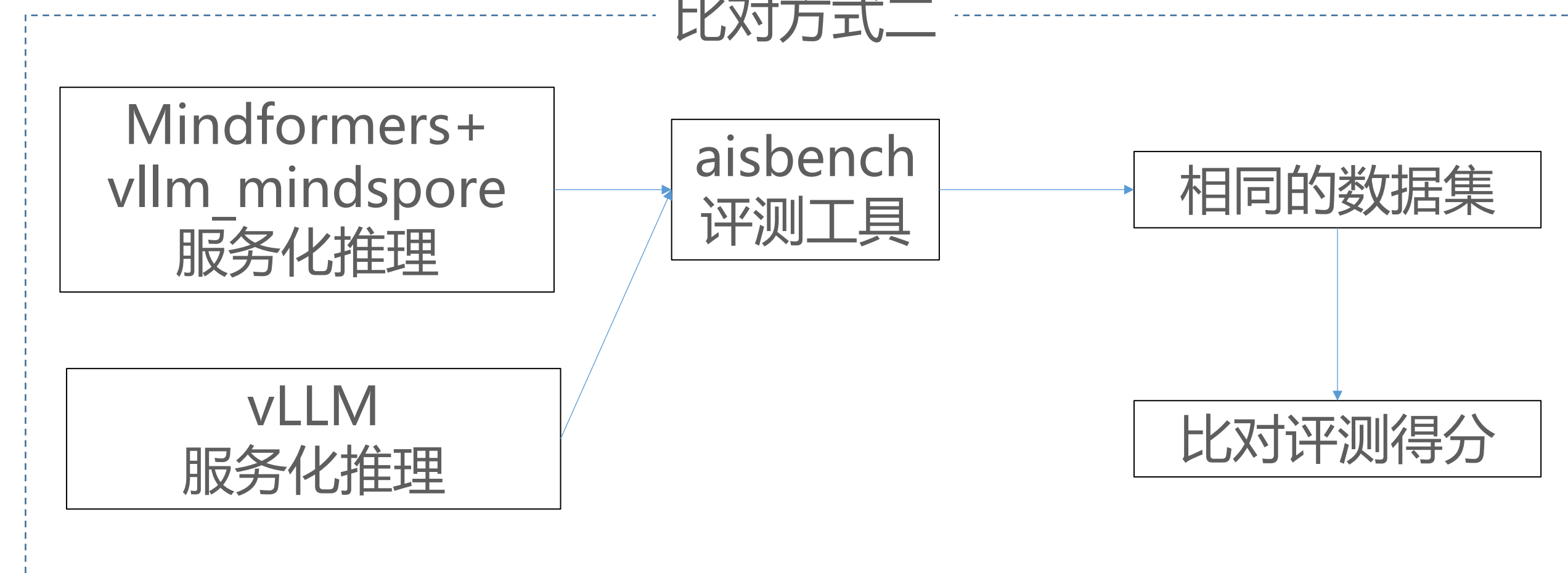
精度比对方式

比对方式一



比对方式一：一般是单条输入，输出比对的重点在于一定长度下token或者logits是否能够对齐

比对方式二



比对方式二：一般是ceval或者gsm8k这种开源数据集，比对的重点在相同评测工具下的评测得分是否在误差范围之内

1. 推理精度对比概念介绍

精度验收标准

模型验收标准

前提：模型的配置，权重，tokenizer都完全一致

比对方式一：相同的单条输入，输出能够和vLLM的前30个token能够完全对齐

比对方式二：相同的评测数据集，对应评测数据集的评测得分能够在0.4%的误差范围之内

接口验收标准

在迁移模型的过程中总会有新结构出现，意味着需要实现新的接口。故，新接口的验收也有一套验收标准。



标准一

```
from tests.utils.double_benchmark import DoubleBenchmarkStandard, DoubleBenchmarkComparator
standard = DoubleBenchmarkStandard(dtype="bfloat16")
DoubleBenchmarkComparator.check_pass_or_not(
    npu_data=npu_data,
    gpu_data=gpu_data,
    golden_data=golden_data,
    standard=standard
)
```

标准二

```
from tests.utils.precision_utils import PrecisionChecker
checker = PrecisionChecker()
checker.check_precision(golden_data, npu_data)
```

1. 推理精度对比概念介绍

精度验收标准

接口验收标准一

双基准精度对比，主要用于在 **NPU（昇腾）、GPU 和 Golden（高精度参考值）** 之间进行浮点计算结果的精度比对，判断算子或模型在不同硬件上的数值一致性。

核心的计算和判断流程如下：

计算小值误差：

$ae = |actual - golden|$

$err_mask = (ae > abs_err_thd) \text{ and } (|golden| < small_value_thd)$

计算正常值误差：

$re = |actual - golden| / clip(|golden|, small_value_thd, None)$

计算均方根误差和误差平衡度：

$$RMSE = \sqrt{1/n \sum (x_i - y_i)^2} \quad \text{-- 均方根误差}$$

$$EB = \frac{1}{n} \sum \frac{x_i - y_i}{\max(|y_i|, 1)} \quad \text{-- 误差平衡度}$$

计算变异系数指标：

$$CV = \frac{|NPU_err|}{\max(|GPU_err|, err_thd)}$$

最后，根据以上流程计算的值和阈值判断是否通过

接口验收标准二

精度检查工具，主要用于在 **NPU（昇腾）和GPU** 之间进行浮点计算结果的精度比对，其通过**余弦相似度**（Cosine Similarity）、**L1归一化偏差**（L1 Norm Ratio）、**KL散度**（KL Divergence）三个指标来判断数值的一致性。

余弦相似度的计算公式：

$$\text{CosSim}(a, b) = \frac{\sum a_i b_i}{\sqrt{\sum a_i^2} \cdot \sqrt{\sum b_i^2}}$$

L1归一化比值偏差的计算公式：

$$\text{L1_Norm} = \frac{\sum |x|}{\sum |y|} - 1$$

KL 散度的计算公式：

$$KL = \sum_i P_{(i)} \log \frac{P_{(i)}}{Q_{(i)}}$$

其中P 和 Q是两个概率分布。

2.推理精度对比流程详述

推理精度对比流程图

环境准备

配置, 权重, Tokenizer对齐

比对纯模型推理结果

比对服务化评测结果

MindSpore
Transformers

使用指定版本

vLLM

使用官方镜像+更新特定
版本vLLM

MindSpore
Transformers

直接复用HF配置, 权
重, Tokenizer

vLLM

使用HF配置, 权重,
Tokenizer

MindSpore
Transformers

纯模型推理

vLLM

离线推理

MindSpore
Transformers

叠加vllm_mindspore
服务化进行评测

vLLM

服务化推理进行评测

环境准备阶段



	MindFormers+vLLM-MindSpore	vLLM
获取途径	参照 官方文档 获取搭建方法	官方文档 获取镜像路径

Step1: 按照官方文档搭建环境后，移除当前环境的mindformers

```
pip uninstall mindformers
```

Step2: 拉取master分支的mindformers

```
git clone https://gitee.com/mindspore/mindformers.git -b master
```

Step3: 设置指定mindformers路径的PYTHONPATH

```
export PYTHONPATH=path/to/mindformers:$PYTHONPATH
```

Step4: 基于当前环境的mindformers迁移适配新模型

Step1: 拉取官方镜像

```
docker pull vllm/vllm-openai:latest
```

Step2: 创建docker容器并进入容器

```
docker run --gpus all -p 8000:8000 \
-v ~/.cache/huggingface:/root/.cache/huggingface \
--env HUGGING_FACE_HUB_TOKEN="your_token_here" \
vllm/vllm-openai:latest \
--model meta-llama/Llama-3-8B \
--tensor-parallel-size 1
```

Step3: 如果当前版本vLLM不符合预期，重装vLLM

```
pip uninstall vllm -y
pip install vllm==x.x.x
```

配置-权重-Tokenizer对齐阶段



	MindFormers+vLLM-MindSpore	vLLM	备注
配置	复用HF的config.json，不需要额外的模型相关配置	使用HF的config.json	MindFormers纯模型推理需要YAML文件启动任务，可以参考 推理配置模板使用指南 开发新模型的YAML文件。
权重	直接读取HF的safetensors格式的权重	使用HF的safetensors格式的权重	关于如何实现直接加载HF的safetensors格式的权重，可以参考 权重转换开发适配 文档实现新模型的权重加载。
Tokenizer	复用的transformers的AutoTokenizer接口，不需要额外的迁移适配	使用的transformers的AutoTokenizer接口	关于如何直接复用HF的tokenizer，可以参考 使用Tokenizer 文档。

比对纯模型推理结果

环境准备

配置，权重，Tokenizer对齐

比对纯模型推理结果

比对服务化评测结果

MindSpore Transformers纯模型推理执行命令

```
# 使用run_mindformer.py 启动一个单卡任务
python run_mindformer.py \
--config {YAML_PATH} \
--run_mode predict \
--use_parallel False \
--predict_data '介绍一下北京故宫'
```

```
# 打印推理任务的输出结果
2025-10-27 10:59:05,742 - xxxx - INFO - output result is:
[{'text_generation_text': ['介绍一下北京故宫的建筑群布局和功能？
\n\n北京故宫的建筑群布局和功能是其重要特色之一，体现了中国古代建筑的智慧和
对自然环境的深刻理解。以下是详细介绍：\n\n1. 建筑群布局：\n北京故宫的
建筑群布局遵循“天人合一”的理念，将自然环境与建筑功能相结合。建筑群分为
中轴线、庭院、宫殿、神']}]
```

去掉问题输入的token，比对前30个token，如果能够对齐，说明纯模型的推理对于这个问题是没有精度问题的；反之，是可能存在精度问题。

该阶段其实在实际的模型迁移过程中，初步搭建完模型，所以最容易出现的精度现象有：

1. 输出乱码或者毫无逻辑
2. 输出不乱码但是前几个token就出现对不齐的情况

vLLM离线推理脚本

```
from vllm import LLM, SamplingParams

MODEL = "path/to/Qwen3-0.6B/"
MAX_TOKENS = 16

question = "帮我指定一份去上海旅游攻略"

sampling_params = SamplingParams(
    max_tokens=MAX_TOKENS,
    temperature=0.0,
)

llm = LLM(
    model=MODEL,
    tensor_parallel_size=1,
    gpu_memory_utilization=0.9,
    trust_remote_code=True,
)

outputs = llm.generate(question, sampling_params)

print("Generated text: ", outputs[0].outputs[0].text)
```

vLLM离线推理执行命令

```
# 启动一个单卡任务
python inference.py
```

```
# 打印推理任务的输出结果
Generated text: 介绍一下北京故宫的建筑群布局和功能？\n\n北京故宫的
建筑群布局和功能是其重要特色之一，体现了中国古代建筑的智慧和
对自然环境的深刻理解。以下是详细介绍：\n\n1. 建筑群布局：\n北京故宫的
建筑群布局遵循“天人合一”的理念，将自然环境与建筑功能相结合。建筑群分为
中轴线、庭院、宫殿、神
```

比对纯模型推理结果

环境准备

配置，权重，Tokenizer对齐

比对纯模型推理结果

比对服务化评测结果

vLLM-MindSpore服务化启动命令

```
# 启动单卡任务
Vllm-mindspore serve ${MODEL_PATH} \
--trust-remote-code \
--max-num-seqs 128 \
--max-model-len 16384 \
--block-size 128 \
--gpu-memory-utilization 0.9 \
--tensor-parallel-size 1
```

vLLM服务化启动命令

```
# 启动单卡任务
python3 -m vllm.entrypoints.openai.api_server \
--model ${MODEL_PATH} \
--trust_remote_code \
--max-num-seqs=128 \
--max_model_len=16384 \
--block-size=128 \
--gpu-memory-utilization=0.9 \
--tensor_parallel_size=1
```

评测

统一使用AISbench进行评测，详细步骤[评测指南](#)文档

Step1 安装AISBench评测环境

Step2 数据集下载

Step3 更改接口配置

Step4 命令行启动评测

该阶段其实在实际的模型迁移过程中，单请求的精度已经对齐。根据验收标准，如果出现精度问题，无非两种情况，可能是如下情况：

1. 虽然评测集得分误差较小，但是依旧满足不了验收标准；
2. 评测集得分误差很大，存在大量请求回答无法对齐vLLM的输出结果

3.推理精度比对问题定位

比对服务化评测结果

输出乱码或者毫无逻辑

[illegible]

比对纯模型推理阶段的精度

环境准备

配置，权重，Tokenizer对齐

比对纯模型推理结果

比对服务化评测结果

输出乱码或者毫无逻辑

1. 权重加载问题

在执行的推理任务的日志中搜索如下日志：

冒号后存在KEY值，部分权重没有加载进网络

```
WARNING - These parameters are not loaded in the network:
{'decoder.layer.x.mlp.experts.weight1'}
```

冒号后不存在任何KEY值，所有权重都加载进网络，但依旧可能存在权重融合或者拆分过程中切分不对导致加载错数据。

```
WARNING - These parameters are not loaded in the network: set()
```

2. 新模型的搭建存在问题

现象：不同的问题推理输出依旧不变。

可能的原因：新网络涉及的MLP模块、MoE模块以及Attention模块等模块的输出存在nan。

定位方法：可以直接打印各个模块的输入输出，观察打印结果是否正常。

解决方法：在确定某个模块出现问题之后，逐算子对比输入输出，直到确定具体某个算子的使用存在问题，然后对齐标杆的修改即可。

输出不乱码但是前几个token就出现对不齐的情况

现象：首Token一致，但是在推了10个token左右就出现精度不一致的现象。

定位方法：一般采用打印和dump数据的方式去对比数据的差异。如果打印的数据无法通过肉眼观察出是否在可接受范围之内，那么可以采用dump数据，然后通过对比工具判定该模块是否符合精度标准。对比工具可以使用MindSpore Transformers提供的方法进行对比，使用方法如下：

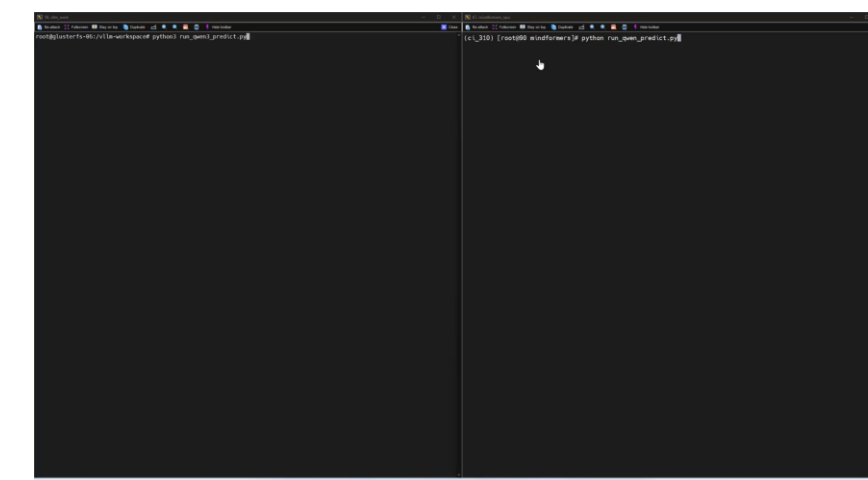
```
import numpy as np
from tests.utils.precision_utils import PrecisionChecker

checker = PrecisionChecker()
gpu_data = np.load('path/to/gpu.npy')
npu_data = np.load('path/to/npu.npy')
checker.check_precision(gpu_data, npu_data)
```

可能的原因：某个输入的dtype类型不一致等导致的精度损失。

解决方法：对齐标杆的dtype。

关于如何dump数据可以参考MindSpore官网提供的[Dump教程文档](#)。



比对纯模型推理阶段的精度

环境准备

配置，权重，Tokenizer对齐

比对纯模型推理结果

比对服务化评测结果

虽然评测集得分误差较小，但是依旧满足不了验收标准

定位方法：找出输出和vLLM回答无法对齐的问题，将问题单独截取出来作为MindFormers纯模型推理和vLLM离线推理的输入，然后按照比对纯模型推理的流程去比对精度。

可能的原因：如果比对纯模型推理结果符合验收标准，大概率是**多batch推理导致精度存在差异**；反之，可能确实存在精度问题。

解决方法：如果是多batch推理的精度存在问题，可以用单batch去评测数据集；如果确实存在精度问题，其问题现象大概率是输出不乱码但是前几个token就对不齐，问题定位思路可以直接**逐模块打印或者dump数据**比对。

评测集得分误差很大，存在大量请求回答无法对齐vLLM的输出结果

定位方法：将大量无法对齐vLLM输出结果的请求——对比，分析无法对齐的情况是否存在相似性。

可能的原因：

1. 如果所有的问题的**输出长度都无法保持一致**，那么基本就是评测使用的接口配置没有对齐，进而导致问题的答案因输出不够长而被判为错误回答；
2. 如果存在一方的回答明显存在模板，而另一方的回答与其不一样，那么基本就是评测使用的接口不一致，进而导致使用的问题的**输入所带有的模板不一致**。

解决方法：对齐评测时的接口和接口配置。

谢谢观看！

MindSpore Transformers & vLLM推理精度比对

MindSpore Transformers 大模型系列课程